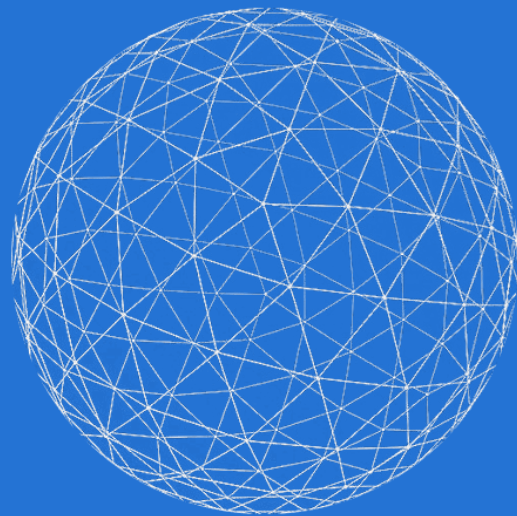




基于2D栅格地图的路径规划与 导航，动态调整与纠偏





CONTENTS

目录

- 1 动态路径规划
- 2 Dijkstra算法
- 3 A*算法
- 4 RRT算法
- 5 DWA算法



教学目标和重难点

➤ 教学目标

1. 了解路径规划的基本概念

重点

2. 了解自动驾驶路径规划常用算法和算法原理

难点

3. 掌握如何选择合适的路径规划算法

难点



① 动态路径规划



动态路径规划

- ◆ ROS建图功能能够解决自动驾驶汽车的定位问题，至于如何从起点行驶到终点，就涉及到路径规划的问题，路径规划可以分为全局路径规划和动态路径规划（也可以叫局部路径规划）。
- ◆ 全局路径规划：全局路径规划算法的目标是在给定障碍物地图、起始点和终点的情况下，找到一条连接起始点和终点且针对障碍物无碰撞的路径。
- ◆ 动态路径规划：动态是指在沿着最优路径前进时，突然出现了既定路径中断的情况需要进行重新规划。



动态路径规划

◆ 现有的全局路径规划算法种类繁多:

- 基于图搜索的dijkstra算法和a star算法等;
- 基于采样的路径规划算法, 如概率路线图(probabilistic roadmaps, 简称prm)算法、prm star算法(prm算法的变种)、快速扩展随机树(rapidly-exploring randomtrees, 简称rrt)算法和rrt*算法(rrt算法的变种)等;
- 基于启发式算法的路径规划算法如蚁群算法和遗传算法等。



② Dijkstra算法

- ◆ Dijkstra 算法，是由荷兰计算机科学家 Edsger Wybe Dijkstra 在1956年发现的算法，戴克斯特拉算法使用类似广度优先搜索的方法解决**赋权图的单源最短路径问题**。
- ◆ Dijkstra 算法原始版本仅适用于找到两个顶点之间的最短路径，后来更常见的变体固定了一个顶点作为源结点然后找到该顶点到图中所有其它结点的最短路径，产生一个最短路径树。
- ◆ 本算法每次取出未访问结点中距离最小的，用该结点更新其他结点的距离。需要注意的是绝大多数的Dijkstra 算法不能有效处理带有负权边的图。



Dijkstra算法

- ◆杰斯特拉算法(Dijkstra)是由荷兰计算机科学家狄克斯特拉于1959年提出的，因此又叫狄克斯特拉算法。是从一个顶点到其余各顶点的最短路径算法，解决的是有权图中最短路径问题。
- ◆迪杰斯特拉算法主要特点是从起始点开始，采用贪心算法的策略，每次遍历到始点距离最近且未访问过的顶点的邻接节点，直到扩展到终点为止。
- ◆这一算法可以用于解决在地图已知情况下的自动驾驶汽车路径规划问题，例如：无人仓储中的自动驾驶汽车路径规划问题、餐厅中上菜机器人路径规划问题等。

◆前面我们已经学习了，利用栅格法，将实际的运动场景加工处理成便于计算机处理的形式，接下来我们用栅格地图讲解Dijkstra算法的工作原理，下图为栅格化的地图，其中深颜色的为障碍物，白色的为可通行区域：

A	B		
C	D		
	E	F	
	G	H	I

- ◆ 现在我们需要自动驾驶汽车规划从A地点到B-I这8个地点的路径，我们通过Dijkstra算法完成这个任务。A-I共9个可达到的节点（右上角节点是不可到达的点，在算法运行过程中不会添加入列表中），我们定义A为起始地点，以A为中心判断其周围的可到达节点：

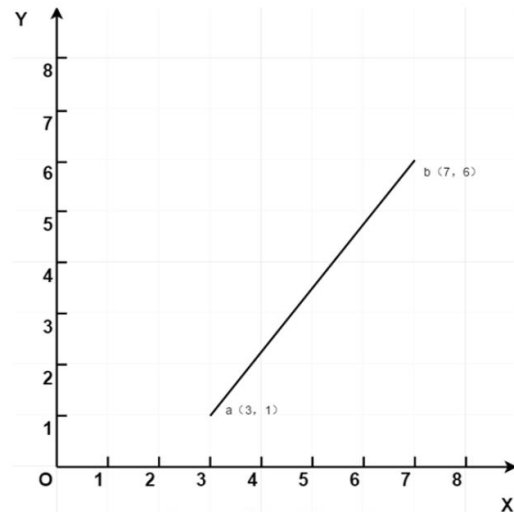
null	null	null
null	A	B
null	C	D

- ◆ 我们可以看到周围的八个节点中，只有B、C、D是可到达的，其余位置都是不存在的（或者说超出图片范围的），然后我们需要对B、C、D三个节点进行进一步的判断，从A到达B、C是可以直接到达的，并且B、C没有被记录（现在只在集合中记录了起始节点A），那么B、C将会被加入到下一次筛选的列表中；从A到D是需要斜着走的，这就要求斜向的路径上不能有阻碍，即B、C不能是障碍物，显然，现在A是可以直接到达D的，那么D也会被记录在下次筛选的备用列表中。

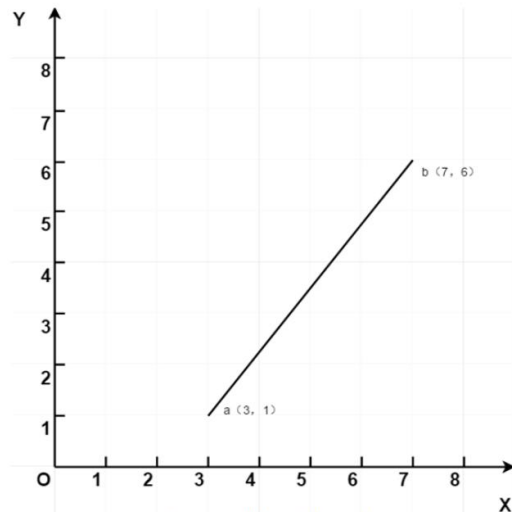
null	null	null
null	A	B
null	C	D

- ◆ 在这里介绍一个关于距离的概念：欧式距离。两个点之间的欧式距离即两点间的直线距离：
- ◆ 例：二维平面两点a(3,1)与b(7,6)间的欧式距离

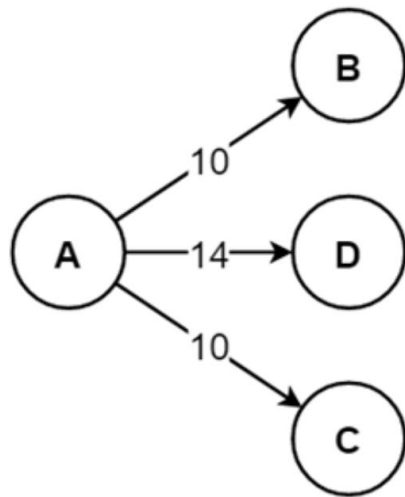
$$\rho = \sqrt{(7-3)^2 + (6-1)^2} = \sqrt{41}$$



- ◆ 我们在图中可以直观地看到从A出发到B、C的距离和到达D的距离是不同的，可以大致估算为10和14 ($10 \times \sqrt{2}$)，这也是需要记录的数据，对于这些要记录的数据，我们分别创建不同的储存空间来记录：
- ◆ 闭集 α ：记录已经确定从起点到达路径的点；
- ◆ 开集 β ：记录已经查找到可到达，但是没有确定路径的点；

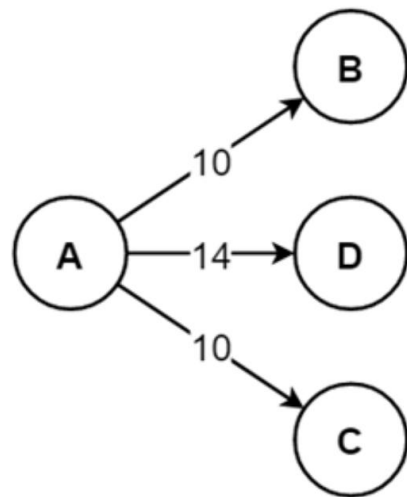


- ◆ 在算法开始时，我们就把起始节点A加入到开集 β 中，在上面的例子中，第一步会把A移动到闭集 α 中，然后对A的周围节点开始检测，然后我们就可以把B、C、D加入到开集 β 中，这时会出现一个问题，我们怎么记录路径呢？
- ◆ 从A检测到并添加到开集 β 的节点有一个共同的特点：从A出发。所以我们可以B、C、D上记录前一步时A，和节点A的距离分别是10、14、10，假设我们并不是在一个4×4的栅格图中规划路径，而是在一个2×2的栅格图中的话，我们只需要将B、C、D加入到闭集 α ，任务就已经完成了，我们可以从每个单元格记录的数据得出，从A直接走到B、C、D就是从A出发的最短路径。



A 节点有效邻域节点距离

◆ 然而我们实际面临的情况是比 2×2 栅格图复杂很多的，在超过 2×2 的栅格图中，我们下一步怎么做呢？我们会在开集 β 中寻找和A距离最近的一个节点，我们只需要比较每个节点记录的数字（B、C是10，D是14），就可以得出B、C离A比较近，然后从中选择一个，将其从开集 β 移动到闭集 α ，然后检测它周围的节点，我们选择移动C节点（实际程序中选择移动B或者C可以自行调整）。



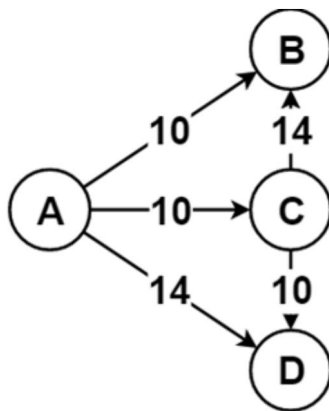
A 节点有效邻域节点距离

- ◆然而我们实际面临的情况是比 2×2 栅格图复杂很多的，在超过 2×2 的栅格图中，我们下一步怎么做呢？我们会在开集 β 中寻找和A距离最近的一个节点，我们只需要比较每个节点记录的数字（B、C是10，D是14），就可以得出B、C离A比较近，然后从中选择一个，将其从开集 β 移动到闭集 α ，然后检测它周围的节点，我们选择移动C节点（实际程序中选择移动B或者C可以自行调整）。
- ◆但是对于B、D节点呢？我们可以比较从C到达B、D的路径距离，可以得出从A路过C到达B、D的距离分别是24、20，是大于现在记录的 distances 的，不会采用这种消耗更大的方式。

null	A	B
null	C	D
null		E

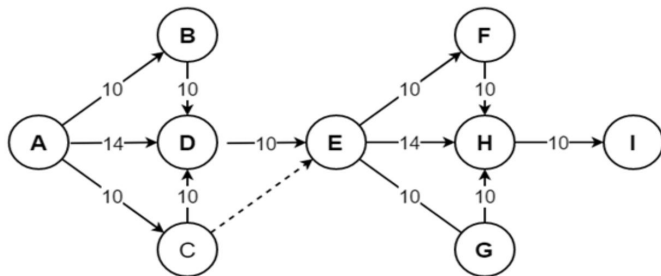
C 节点邻域分析

◆ 假设得出的是优于现在记录的路径，则会修改该点记录的上一步节点和距离数值，例：B (C, 24)



C 节点有效邻域节点距离

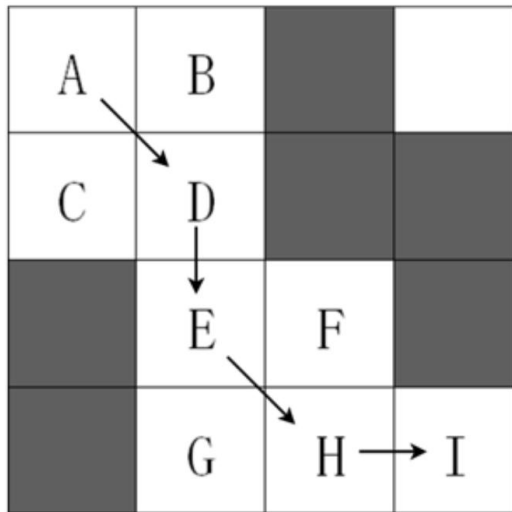
◆ 重复上述步骤，我们可以得出从A到每个可到达点之间的每一步距离，如图（C不可到达E）：



A 节点到达 I 节点距离分析

◆ 从而得出从A出发到达每个可到达点的距离，路径可以利用每个点记录的上一步节点名通过一步一步回溯得出，从I开始回溯的步骤：I的上一步是H，H的上一步是E，E的上一步是D，D的上一步是A

◆ 最后可得出从A到I的最优路径如下图所示：





2 A^* 算法

- ◆ Astar算法是一种静态路网中求解合适路径最有效的启发式搜索算法，也是解决许多搜索问题的有效算法。和Dijkstra算法相比，Astar 算法的查找效率更高，能够更快地找到从起始点到达终点的路径，Astar算法得出的路径和Dijkstra算法得出的路径是完全相同的。
- ◆ 相比于Dijkstra算法的每走一步，观察周围，试探所有可行路径的方式，Astar算法增加了一步——利用代价函数确定搜索方向，明显缩减了节点查询的运算量。
- ◆ 在Astar中我们引入代价函数，来判断下一步需要到达的哪个节点，我们根据如下代价函数：

$$F = G + H$$

◆在Astar中我们引入代价函数，来判断下一步需要到达的哪个节点，我们根据如下代价函数：

$$F = G + H$$

其中G：表示从起始节点到达当前节点的路径代价

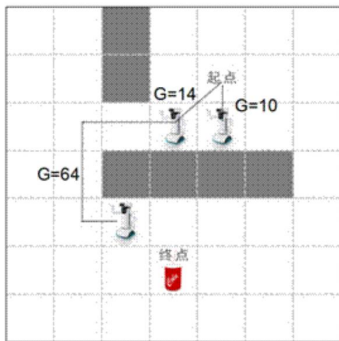


图3 G值的计算方法

$$F = G + H$$

其中H：表示从当前节点到终点的曼哈顿距离

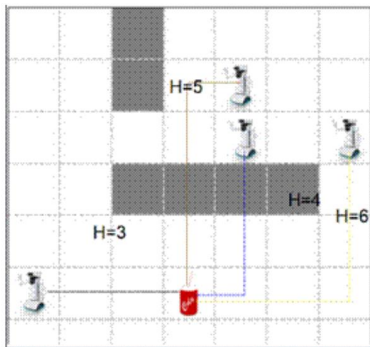


图4 H值的计算方法

◆计算G与H我们分别使用欧氏距离与曼哈顿距离进行计算，下面介绍新的代价函数计算方式：曼哈顿距离。

◆ 二维平面两点 $a(x_1, y_1)$ 与 $b(x_2, y_2)$ 间的曼哈顿距离：

$$D_{12} = |x_2 - x_1| + |y_2 - y_1|$$

例：二维平面两点 $a(3, 1)$ 与 $b(7, 6)$ 间的曼哈顿距离：

$$D_{ab} = |3 - 7| + |1 - 6| = 9;$$

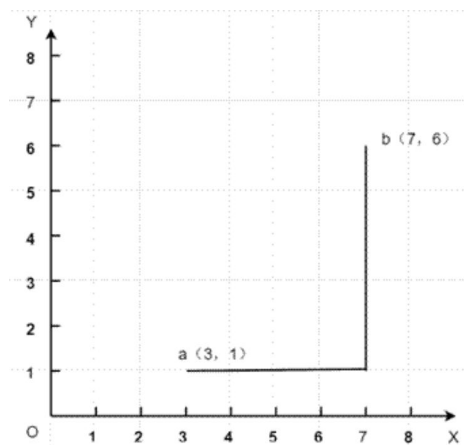


图5 曼哈顿距离示例

- ◆ 由于终点的位置是已知的，所以每次规划下一步路径时，本次节点周围的八个节点与终点的曼哈顿距离是可知的（不计后续路径中可能出现的障碍物），所以在路径规划过程中，不必尝试每个节点（Dijkstra算法的方式），只要向终点方向（与终点距离最短的节点）走下一步即可，相比于Dijkstra算法无方向性的层层扩展方式，显然Astar向目标前进的方式效率更高。
- ◆ 在算法的实现过程中，我们依旧会建立两个集合来储存查找过程中得到的信息：
 - 闭集 α ：记录已经确认与起始点最短路径的点；
 - 开集 β ：记录已经找到但是未确认与起始点最短路径的点；

- ◆在算法开始时，在开集 β 中添加起始节点A；
- ◆首先我们会查找开集 β 中F最小的点（此时开集 β 中只有起始点A），在找到这个点后，我们将其移动到闭集 α 中，此时以A为中心，对其周围的八个节点进行检测。首先需要确定判断每个节点是否可达，可达的节点需要满足以下条件：

- 1. 不是障碍物
- 2. 不在闭集 α 中
- 3. 未超出地图
- 4. 直接可达（这一条件主要针对斜向的节点，需要斜向对应的方向相邻两节点都是可通过的，例如A $\rightarrow$ D可行，D $\rightarrow$ F不可行）；

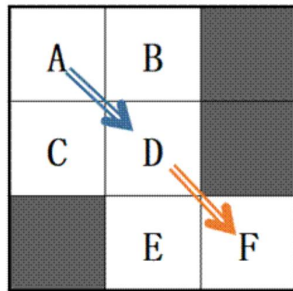


图6 A $\rightarrow$ D无阻碍，D $\rightarrow$ F有阻碍



A*算法

- ◆ 完成上述判断后，对可达的点（即有效邻域节点）添加到开集 β 中，并对这些有效节点的G值、H值和F值进行计算；如果该点已经在开集中，则比较已经记录的G值和途径当前节点到达该点的G值，将其父节点记录更改为G值较小的路径（相等不修改）。
- ◆ 重复上述步骤直至终点出现在开集 β 中，从终点依次按照记录的父节点回溯得出规划的路径；如果直至开集 β 为空都没有找到终点，则返回空，或提示未找到路径。

◆在算法运行过程中，需要注意处理两个问题：

1. 开集中存在一个以上相同最小F值的节点，如何选取下一步节点。

在新的节点加入到开集 β 时，我们会利用C++的指针功能，设定一个指针始终指向开集 β 中F值最小的节点，当加入一个新的节点时，我们会将其F值与当前指针指向的节点的F值相比较，如果当前指向的节点F值较小，则不做改变；如果新加入节点的F值较小，则指向新介入的节点；如果新加入节点的F值与当前指向的节点相等，我们可以自行选择不做改变或者指向新的节点，下面例程中选择指向新加入的节点。

2. 如何利用G值更新路径

在每次检测当前节点为中心的八个节点时，如果节点可达，可以得出从起始节点到达节点的距离：起始节点到当前中心节点的距离和中心节点到达当前点的距离之和。如果该节点不在开集 β 中，则记录本次得到的G值并将其加入到开集 β ；如果节点已经在开集 β 中，表示之前已经找到另外一条到达这个节点的路径，此时开集 β 中会记录另外路径的G值，与当前得到的G值相比较，G值较小的是较短的路径，选择将节点的父节点记录为G值较小的路径的上一步节点，更新信息即可。

◆我们使用4×4栅格图来模拟算法具体步骤

在图7中，我们设置起点为A，终点为I，从起始点A出发，查找到附近有效邻域节点B、C、D后，我们分别从B、C、D处向终点I的方向“看”一下，得到H值（此处采用欧式距离示例），并且将H值和G值相加，可以得到结果：B、C的F值为： $10 + \sqrt{20^2 + 30^2} = 10 + 10\sqrt{13} \approx 46$ ，而D的F值为： $\sqrt{30^2 + 30^2} = 30\sqrt{2} \approx 42$ ，比较可以发现从A出发途径D到达I的总距离是比较短的，在这种情况下，下一步的起始节点就会是D，以D为中心我们可以发现新的有效节点E，我们从A到E的查找只经历了两次A→D、D→E，而使用Dijkstra算法，我们需要查找判断A→B、A→C、A→D、C→B、C→D、B→D、D→E七次，显然A*算法是更高效的。

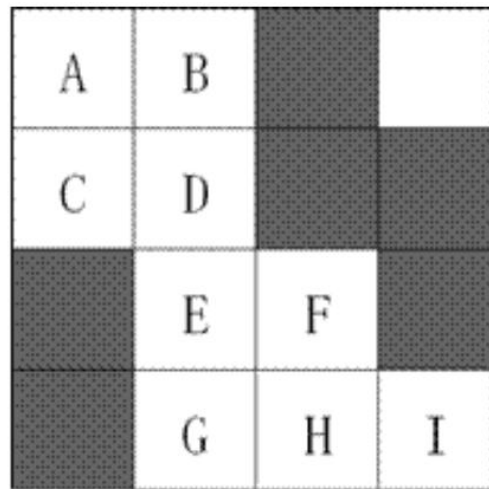


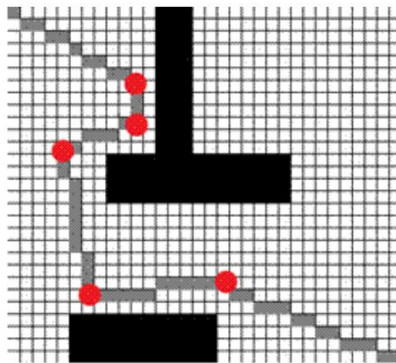
图7 Astar原理实验例图

◆相比于Dijkstra算法，Astar更加充分的利用了全部地图已知的信息，在Dijkstra算法“视野”局限于当前节点周围的情况下，寻找终点的任务执行过程表现为从起始点开始，获取起始点周围的所有信息并全部尝试，直到“遇到”终点；而Astar算法则“视野”变得更加开阔，每走一步都会“望一望”，每次“寻找”都能趋向是离终点更近的，是一种“找到”终点的方式。



③ RRT算法

- ◆ 快速扩展随机树 (Rapidly-exploring Random Trees-----RRT) 算法由美国爱荷华州立大学的Steven M.LaValle教授在1998年提出的，是目前广泛应用的基于采样的运动规划算法。
- ◆ RRT算法的思想是创建一个从起始点随机生长的树，树的生长方向是会趋于空白区域的，在障碍物较少的情况下，使用RRT可以很快的探索空间，找到通往目标节点的路径，但是这个路径不会是最优的。图中的可以看出，RRT规划的路径是存在折返的情况的，这是由于每一步都是随机生成，以图3举例，其节点分布如下图：



RRT规划路径节点位置

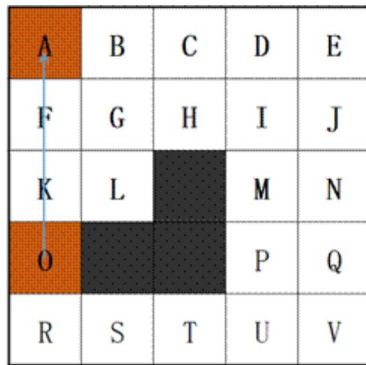
- ◆ RRT在运行前也会读取地图，但是RRT对地图的处理和之前讲过的Dijkstra算法与AStar算法不同，RRT会首先遍历每一个栅格，将所有非障碍物的栅格记录在一个新的列表中（Dijkstra和AStar是建立对应的二维数组记录每个栅格是否障碍物），此列表下面称为备选节点列表。
- ◆ 算法开始输入起始节点和终点后，通过随机的方式，在备选节点列表中选取一个点，然后在随机树上找一个与其距离最短的点（第一次运行时随机树只有起始节点这个根节点存在），然后尝试连接两点，如果连接两点的直线上没有障碍物，就可以认为当前选中的点是可以作为树上的节点的，此时将选中节点的父节点记录为树上距离它最近的点，对应到树，可以认为这一次的“枝”是由“树”上某一点“生长”达到的。重复“生长”的过程，直到“树”的节点到达目标，或者到达可以直接到达目标节点的位置，算法完成，回溯节点得出路径。

- ◆假设我们要从A到达V，显然是不能直接直线到达的，使用RRT算法规划一条路径。首先将可到达格子（A - V）添加到备选节点列表，设置A为起始节点，V为目标节点。将A作为随机树的根节点，从备选列表中随机选取节点。假设第一次选取结果为O，显然O是可以直接与A连接的，将O从备选节点列表移除，并将父节点设置为A并将O加入到随机树，观察可得，O与V之间是有障碍物的，并不能直接到达V；

A	B	C	D	E
F	G	H	I	J
K	L		M	N
O			P	Q
R	S	T	U	V

节点O加入随机树

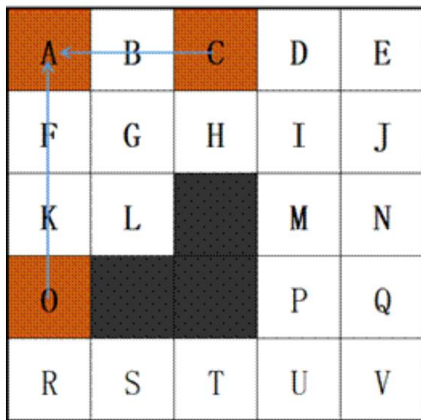
- ◆ 假设我们要从A到达V，显然是不能直接直线到达的，使用RRT算法规划一条路径。首先将可到达格子（A - V）添加到备选节点列表，设置A为起始节点，V为目标节点。将A作为随机树的根节点，从备选列表中随机选取节点。假设第一次选取结果为O，显然O是可以直接与A连接的，将O从备选节点列表移除，并将父节点设置为A并将O加入到随机树，观察可得，O与V之间是有障碍物的，并不能直接到达V；
- ◆ 然后我们可以继续从备选节点列表选取新的点，假设下一次获得的点是M，我们可以发现M距离树中的点O较近，但是M与O中间是有障碍物的，这时选择将M放回备选节点列表中，对随机树不做改动。



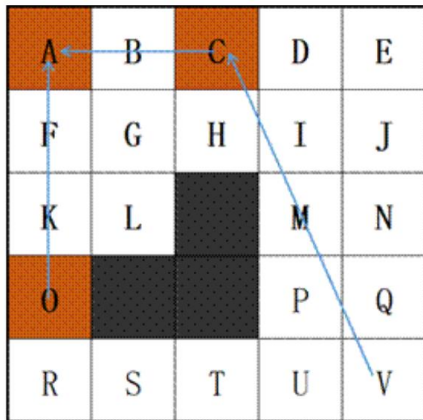
节点O加入随机树

- ◆ 假设下次选取到节点C，我们对比树中记录的信息，可以发现C距离树中的节点A较近，并且C是可以直接到达A的，我们将C从备选节点列表移除，将父节点设置为A并加入到随机树中（图5）。此时我们尝试从C直接到达V，直观来看是没有障碍物的，但是计算机怎样识别有没有障碍物呢，这就需要我们设计一个函数来做判断。
- ◆ 首先，我们需要先获取C到V的欧氏距离，此处可以简单计算得到距离 $d = \sqrt{2^3 + 4^2} = 2\sqrt{5} \approx 4$ ，我们可以认为C到V需要走四步，走完四步我们就可以到达V，那么C与V之间就有三个节点，如果这三个节点都不是障碍物，计算机就可以认为C是可以直接连通到V的。但是怎么确定这三个节点的位置呢？我们可以把C与V用直线段相连，并把该直线段均分为四等分，然后判断每一等分的中点在哪个栅格中，我们就认为通过了哪个节点，最后一段的中点就在目标节点V内。

- ◆ 我们可以发现节点C与目标节点V之间是没有障碍物的，可以认为C可以直接走到V，RRT算法得到了从A到达V的路径（图7）：A→B, B→C, C→I, I→M, M→Q, Q→V。



节点C加入随机树



RRT得出从A到V路径

- ◆ 由上例可发现，RRT规划路径的过程是比Dijkstra和AStar简单的，但是规划的结果并不比Dijkstra和AStar更优，在三维空间做路径规划时，使用Dijkstra和AStar必须要对运动空间进行建模（栅格图转化二维矩阵），这一步代价是比较大的，而使用RRT是可以不建模的，取随机点后我们只需要对树的拓展进行碰撞检测，就可以快速完成树的扩展找到可行的路径。



③ DWA算法

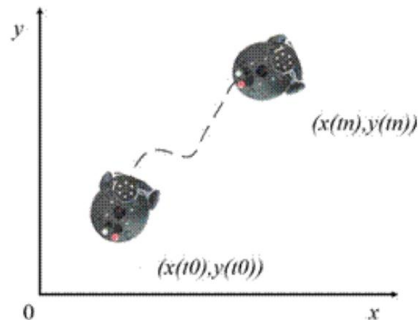
- ◆ Fox 与 Thrun 等人在 1997 年提出了自动驾驶汽车局部避障的动态窗口法(Dynamic Window Approach, DWA)。动态窗口法 DWA 是在约束的速度矢量空间内采样线速度与角速度作为代价点，之后运用自动驾驶汽车运动学计算其待代价轨迹，之后对待代价轨迹进行评分选取最优轨迹，保证了自动驾驶汽车运动速度命令的可行性。
- ◆ 动态窗口法 DWA 的实现包含两个步骤：
 - 对自动驾驶汽车速度进行约束限制，形成动态窗口进行速度采样；
 - 根据速度采样点求出待代价轨迹，最大化代价函数（或最小化代价函数）选取合适路径对应速度命令。
- ◆ 动态窗口是由一系列的约束构成，其中约束主要包括差速自动驾驶汽车的非完整约束、环境障碍物约束与受结构与电机影响的动力学约束。

◆自动驾驶汽车底盘的运动模型

下图所示为差速自动驾驶汽车运动学模型示意图， v_l 与 v_r 代表自动驾驶汽车底盘左右轮的线速度， $v(t)$ 和 $w(t)$ 分别代表自动驾驶汽车在世界坐标系下的平移速度与转动速度， L 代表自动驾驶汽车左右轮距。

$$v(t) = \frac{(v_l(t) + v_r(t))}{2}$$

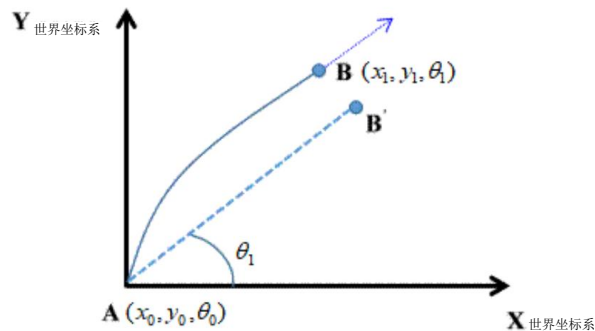
$$w(t) = \frac{(v_r(t) - v_l(t))}{L}$$



差速机器人运动学模型示意图

◆自动驾驶汽车底盘的运动模型

因为自动驾驶汽车在编码器采样周期内位移较小，所以在相邻两个时间采样点内将自动驾驶汽车运动轨迹近似为直线，即自动驾驶汽车沿着其航向角方向平移 $v(t)\Delta t$ 之后其航向角相对于世界坐标系转动 $w(t)\Delta t$ 。将该段距离与角度转换到世界坐标系下即为自动驾驶汽车的轨迹。则自动驾驶汽车运动轨迹可由如下简图与公式表示：



差速机器人运动学模型示意图

◆自动驾驶汽车底盘的运动模型

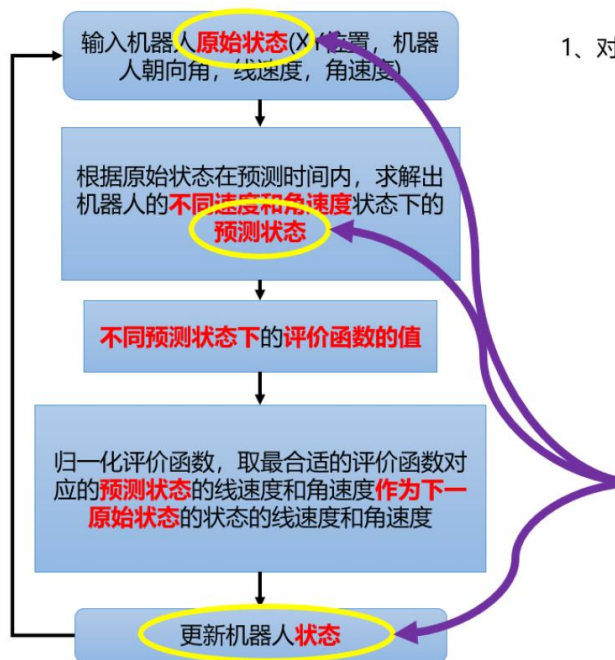
A点设为当前点，B点为下一个轨迹点，通过A点的位姿信息与时间间隔，计算B点的位姿信息，我们采用以直代曲的思想，用B'，公式如下：

$$x_1 = x_0 + v \cos \theta_t dt$$

$$y_1 = y_0 + v \sin \theta_t dt$$

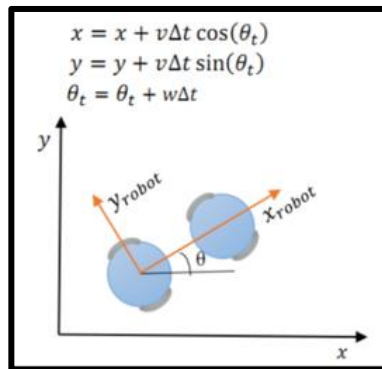
$$\theta_1 = \theta_0 + w dt$$

◆DWA路径规划流程

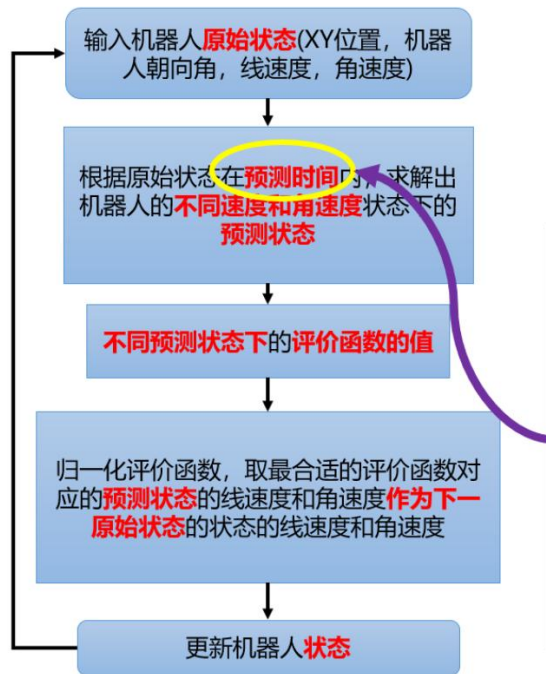


1、对于机器人状态求解, 采用下面的模型, 按照矩阵形式即

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} X \\ Y \\ \theta \\ V \\ W \end{bmatrix} + \begin{bmatrix} dt * \cos(\theta) & 0 \\ dt * \sin(\theta) & 0 \\ 0 & dt \\ 1 & 0 \\ 0 & 1 \end{bmatrix} * \begin{bmatrix} V \\ W \end{bmatrix}$$



◆DWA路径规划流程

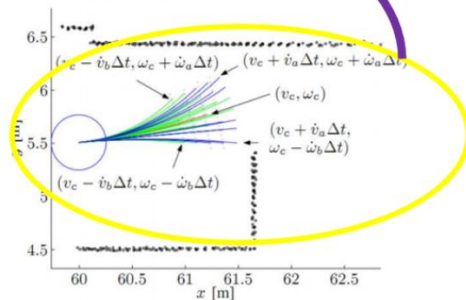


2、对于预测时间和当前预测下的评价函数关系

V和W在预测范围内都有对应 $V_{min} \rightarrow V_{max}$ 和 $W_{min} \rightarrow W_{max}$ 两两组合的 V_i, W_i , 二者决定了不同的评价函数, 决定机器人下一状态以 V_j, W_j 的方式进行运动

同时注意: 为了简化每组速度对应轨迹的计算, 该算法假设机器人在往前模拟轨迹的这段时间(sim_period)内速度不变, 直到下一时刻采样给定新的速度命令。

动态窗口采样的轨迹如下图所示:



◆DWA路径规划流程



2、对于预测时间和当前预测下的评价函数关系

因此一个原始状态到预测状态的过程存在n多个选择, 如何选择则通过最后的 $G(V,W)$ 进行决定

在采样的速度组中, 有若干组轨迹是可行的, 因此采用评价函数的方式为每条轨迹进行评价。在原始论文中, 采用的评价函数如下:

$$G(v, \omega) = \sigma(\alpha \cdot \text{heading}(v, \omega) + \beta \cdot \text{dist}(v, \omega) + \gamma \cdot \text{velocity}(v, \omega))$$

存放预测时间内不同的Vti	存放预测时间内不同的Wti	存放由前面Vti和Wti得出的heading	存放由前面Vti和Wti得出的dist	存放由前面Vti和Wti得出的velocity	存放评价函数归一化且加权的值Gi(Vti,Wti)
---------------	---------------	------------------------	---------------------	-------------------------	---------------------------

.....

存放预测时间内不同的Vtn	存放预测时间内不同的Wtn	存放由前面Vtn和Wtn得出的heading	存放由前面Vtn和Wtn得出的dist	存放由前面Vtn和Wtn得出的velocity	存放评价函数归一化且加权的值Gn(Vtn,Wtn)
---------------	---------------	------------------------	---------------------	-------------------------	---------------------------

◆DWA路径规划流程



4、归一化和防止冲突

3.4 平滑处理

也就是**归一化**, 上面三个部分计算出来以后不是直接相加。而是每个部分在归一化以后, 再相加。

如何归一化?

每一项除以每一项的总和:

$$normal_head(i) = \frac{head(i)}{\sum_{i=1}^n head(i)}$$

$$normal_dist(i) = \frac{dist(i)}{\sum_{i=1}^n dist(i)}$$

$$normal_velocity(i) = \frac{velocity(i)}{\sum_{i=1}^n velocity(i)}$$

其中, n 为采样的所有轨迹, i 为待评价的当前轨迹。

为什么要归一化?

归一化的目的是 **smoothing**: 譬如障碍物距离, 机器人传感器检测到的最小障碍物距离在二维空间中是不连续的, 这条轨迹能够遇到障碍, 旁边那边不一定能遇到。并且这条轨迹最小的障碍物距离是 1m, 旁边那条就是 10m。那么障碍物距离的这种评价标准导致评价函数不连续, 也会导致某个项在评价函数中太占优势, 如这里的离障碍物距离 10m 相对于 1m 就太占优势。所以将它们归一化:

$$normal_dist(i) = \frac{dist(i)}{\sum_{i=1}^n dist(i)}$$

这样, 都变成同一的百分比了, 每个障碍物最小距离都是这 100 份中的一份。

总结起来三者构成的评价函数的物理意义是: 在局部导航过程中, 使得机器避开障碍, 朝着目标以较快速度行驶, 缺一不可。

- ◆全局路径规划是依据给定的目标，接受权值地图信息生成全局权值地图，规划出从起点到目标位置的全局路径，作为局部路径规划的参考。本章介绍的Dijkstra算法，A*算法和RRT算法，都属于全局路径规划算法。
- ◆局部路径规划层是作为导航系统的局部规划部分，接受权值地图生成的局部权值地图信息，依据附近的障碍物信息进行局部路径规划。本章介绍的DWA算法属于局部路径规划算法。



Thank You

AI改变生活